



Configuración de seguridad para la base de datos empresa-retail

Emerson Andrés Medina Pencue

Fundación Universitaria de Popayán

Especialización de Data Analytics para Marketing Digital

Administración de Base de Datos

Ing. Luis Vejarano

Octubre de 2026

Tabla de contenido

Introducción	3
Objetivos	3
Marco conceptual.....	4
Control de versiones con Git y GitHub.....	4
Seguridad de acceso en MySQL	4
Desarrollo de la actividad	5
1. Creación del repositorio con las ramas main y develop	5
2. Incorporación del archivo README.md	7
3. Incorporación del archivo .gitignore para SQL	8
4. Solución de la actividad en la rama develop.....	9
Análisis y recomendaciones de seguridad	12
Conclusión	13
Referencias.....	15

Lista de figuras y tablas

Figura 1. Creación del repositorio empresa-retail-admin en GitHub	6
Figura 2. Archivo README.md en la rama main	8
Figura 3. Contenido de la rama develop del repositorio	9
Figura 4. Script SQL de creación de usuarios y asignación de permisos	10
Figura 5. Verificación de los usuarios en MySQL Workbench.....	12
Tabla 1. Comandos de Git utilizados en el flujo de trabajo.....	7
Tabla 2. Resumen de usuarios, objetos y privilegios asignados	11

Introducción

La información almacenada en una base de datos suele ser uno de los activos más valiosos de una organización, por lo que su protección es una de las tareas centrales de la administración de bases de datos. Proteger la información no consiste solamente en impedir accesos externos, sino también en controlar qué puede hacer cada persona que sí está autorizada a conectarse: quién puede consultar un dato, quién puede modificarlo y quién únicamente debe auditarlo. Este control se logra mediante la creación de cuentas de usuario y la asignación precisa de privilegios.

Este documento técnico describe la actividad realizada para configurar la seguridad de la base de datos “empresa-retail”, cuyo esquema agrupa las tablas canal, cliente, conversion e interaccion. La configuración divide las responsabilidades entre el personal de Cajas, Inventario y Gerencia mediante tres usuarios de MySQL con permisos diferenciados, aplicando el principio de mínimo privilegio (Saltzer y Schroeder, 1975). Para garantizar la trazabilidad de los cambios, todo el material se gestionó con el sistema de control de versiones Git y se publicó en la plataforma GitHub, en el repositorio “empresa-retail-admin”.

El documento se organiza así: primero se presentan los objetivos y el marco conceptual; luego se detalla el desarrollo de las cuatro tareas solicitadas (creación del repositorio con las ramas *main* y *develop*, incorporación del archivo README.md, incorporación del archivo .gitignore para SQL y entrega de la solución en la rama *develop*); después se analizan aspectos de seguridad y, finalmente, se presentan la conclusión y las referencias. El formato sigue las normas de la séptima edición del *Manual de publicaciones de la American Psychological Association* (American Psychological Association [APA], 2020).

Objetivos

Objetivo general

Configurar la seguridad de la base de datos “empresa-retail” mediante la creación de usuarios y la asignación de permisos diferenciados, documentando y versionando el trabajo en un repositorio de GitHub.

Objetivos específicos

- Crear el repositorio “empresa-retail-admin” con las ramas *main* y *develop*.
- Incorporar un archivo README.md con la descripción del proyecto y un archivo .gitignore adecuado para proyectos SQL.
- Crear los usuarios ana_crm, pedro_mkt y marta_auditoria en MySQL.
- Asignar a cada usuario únicamente los permisos que requiere su función.
- Almacenar la solución en la rama *develop* y elaborar el presente documento técnico.

Marco conceptual

Control de versiones con Git y GitHub

Git es un sistema de control de versiones distribuido que registra los cambios realizados sobre los archivos de un proyecto y permite trabajar en líneas de desarrollo paralelas (Chacon y Straub, 2014). Una *rama* es una de esas líneas: permite desarrollar y probar cambios sin afectar la versión estable del proyecto, que habitualmente se conserva en la rama *main* (GitHub, s. f.-a). GitHub es la plataforma que aloja repositorios Git en la nube y facilita su consulta y colaboración.

Dos archivos acompañan habitualmente a un repositorio. El archivo README.md presenta el proyecto a quienes lo visitan y GitHub lo muestra automáticamente en la página principal (GitHub, s. f.-b). Por su parte, el archivo .gitignore indica a Git qué archivos o carpetas

no deben versionarse, como copias de respaldo, registros o configuraciones locales que puedan contener credenciales (Git, s. f.; GitHub, s. f.-c).

Seguridad de acceso en MySQL

En MySQL, cada cuenta se identifica por un nombre de usuario y un host de conexión, y se crea con la sentencia `CREATE USER` (Oracle Corporation, s. f.-a). Los permisos se otorgan con la sentencia `GRANT`, que puede aplicarse a nivel global, de base de datos, de tabla, de columna o de rutina almacenada (Oracle Corporation, s. f.-c). Los privilegios más usados en este trabajo son `SELECT` (consultar), `INSERT` (insertar), `UPDATE` (actualizar), `DELETE` (eliminar) y `EXECUTE` (ejecutar procedimientos almacenados) (Oracle Corporation, s. f.-d).

El criterio que guía la asignación es el principio de mínimo privilegio: cada usuario debe operar con el menor conjunto de permisos necesario para cumplir su función, de modo que un error o un uso indebido tenga el menor impacto posible (Saltzer y Schroeder, 1975). A partir de la versión 8.0, MySQL permite además agrupar privilegios en roles para simplificar su administración (Oracle Corporation, s. f.-e).

Desarrollo de la actividad

Esta sección describe cada tarea solicitada y presenta las evidencias capturadas durante su ejecución. Todas las figuras son capturas de pantalla de elaboración propia.

1. Creación del repositorio con las ramas *main* y *develop*

El primer paso fue crear en GitHub un repositorio público llamado “empresa-retail-admin”, a nombre de la cuenta Andresmedina2210. Durante la creación se activó la opción de agregar un archivo `README`, y GitHub generó de forma automática la rama *main*, que actúa como rama predeterminada (GitHub, s. f.-d). La Figura 1 muestra el formulario de creación con el nombre del repositorio, la visibilidad pública y las opciones de configuración seleccionadas.

Figura 1

Creación del repositorio empresa-retail-admin en GitHub

Create a new repository
 Repositories contain a project's files and version history. Have a project elsewhere? [Import a repository](#).
 Required fields are marked with an asterisk (*).

1 General

Owner * Andresmedina2210 **Repository name *** empresa-retail-admin
 Checking availability...
 Great repository names are short and memorable. How about [stunning-ecto-journey](#)?

Description
 0 / 250 characters

2 Configuration

Choose visibility * Public
 Choose who can see and commit to this repository

Add README
 READMEs can be used as longer descriptions. [About READMEs](#) ☒ On

Add .gitignore
 .gitignore tells git which files not to track. [About .gitignore files](#)

Add license
 Licenses explain how others can use your code. [About licenses](#)

Nota. Captura de pantalla de elaboración propia del formulario *Create a new repository* de GitHub. Se observa el propietario (Andresmedina2210), el nombre del repositorio (empresa-retail-admin), la visibilidad pública y la opción *Add README* activada. Fuente: GitHub (<https://github.com/new>).

Posteriormente se clonó el repositorio en el equipo local y se trabajó sobre la rama *develop*, que funciona como espacio de integración: allí se desarrolla la solución antes de incorporarla a *main*, lo que evita alterar la versión estable mientras el trabajo está en curso (Chacon y Straub, 2014; GitHub, s. f.-a). La Tabla 1 resume los comandos empleados en este flujo de trabajo.

Tabla 1*Comandos de Git utilizados en el flujo de trabajo*

Comando	Propósito	Resultado esperado
<code>git clone <URL></code>	Copiar el repositorio remoto al equipo local.	Carpeta local con la rama <i>main</i> .
<code>git checkout develop</code>	Cambiar a la rama de trabajo (con <code>-b</code> si aún no existe).	Rama activa: <i>develop</i> .
<code>git add <archivo></code>	Preparar los archivos que se van a registrar.	Archivos en el área de preparación.
<code>git commit -m "mensaje"</code>	Registrar los cambios con un mensaje descriptivo.	Nueva confirmación en <i>develop</i> .
<code>git push -u origin develop</code>	Enviar la rama <i>develop</i> a GitHub.	Rama publicada en el repositorio remoto.

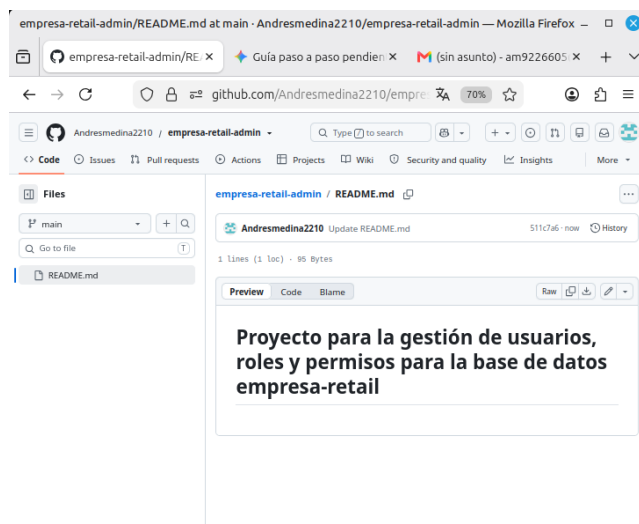
Nota. Elaboración propia con base en Chacon y Straub (2014).

2. Incorporación del archivo README.md

El archivo README.md contiene la descripción del proyecto y es lo primero que ve quien visita el repositorio (GitHub, s. f.-b). En este caso se editó en la rama *main* para incluir el título “Proyecto para la gestión de usuarios, roles y permisos para la base de datos empresa-retail”, tal como se aprecia en la Figura 2. GitHub registró el cambio como una nueva confirmación con el mensaje *Update README.md*.

Figura 2

Archivo README.md en la rama main



Nota. Captura de pantalla de elaboración propia del archivo README.md del repositorio empresa-retail-admin, en la rama main. Se muestra la vista previa con el título del proyecto y el registro de la última confirmación. Fuente: GitHub.

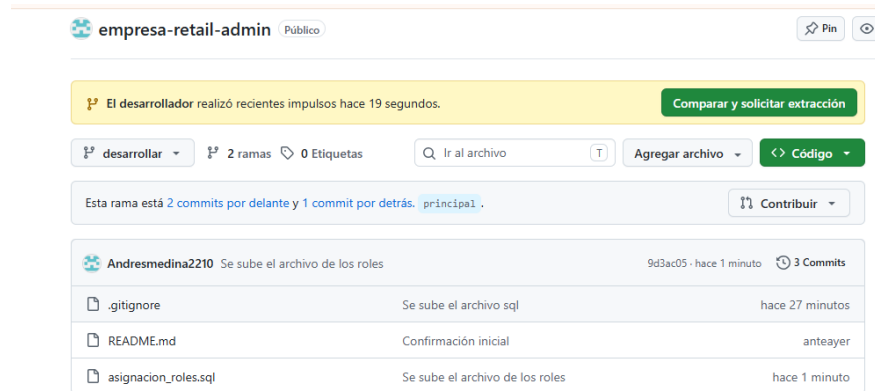
3. Incorporación del archivo .gitignore para SQL

Se creó en la rama *develop* un archivo `.gitignore` orientado a proyectos de bases de datos. Este archivo evita que Git versione elementos generados de forma local o sensibles (Git, s. f.). Para este proyecto se excluyeron las bases de datos locales (`*.sqlite`, `*.sqlite3`, `*.db`), las copias de respaldo y volcados (`*.bak`, `*.dump`, `*.sql.gz`), los registros (`*.log`), los archivos de credenciales y configuración (`.env`, `*.cnf`), los directorios de editores (`.vscode/`, `.idea/`) y los archivos del sistema operativo (`.DS_Store`, `Thumbs.db`). Los archivos con extensión `.sql` no se ignoraron, porque constituyen la solución que debe versionarse.

El archivo se registró con el mensaje de confirmación “Se sube el archivo sql” y se envió a GitHub con `git push -u origin develop`. La Figura 3 muestra la rama *develop* con el archivo `.gitignore`, el archivo README.md y el script de roles.

Figura 3

Contenido de la rama develop del repositorio



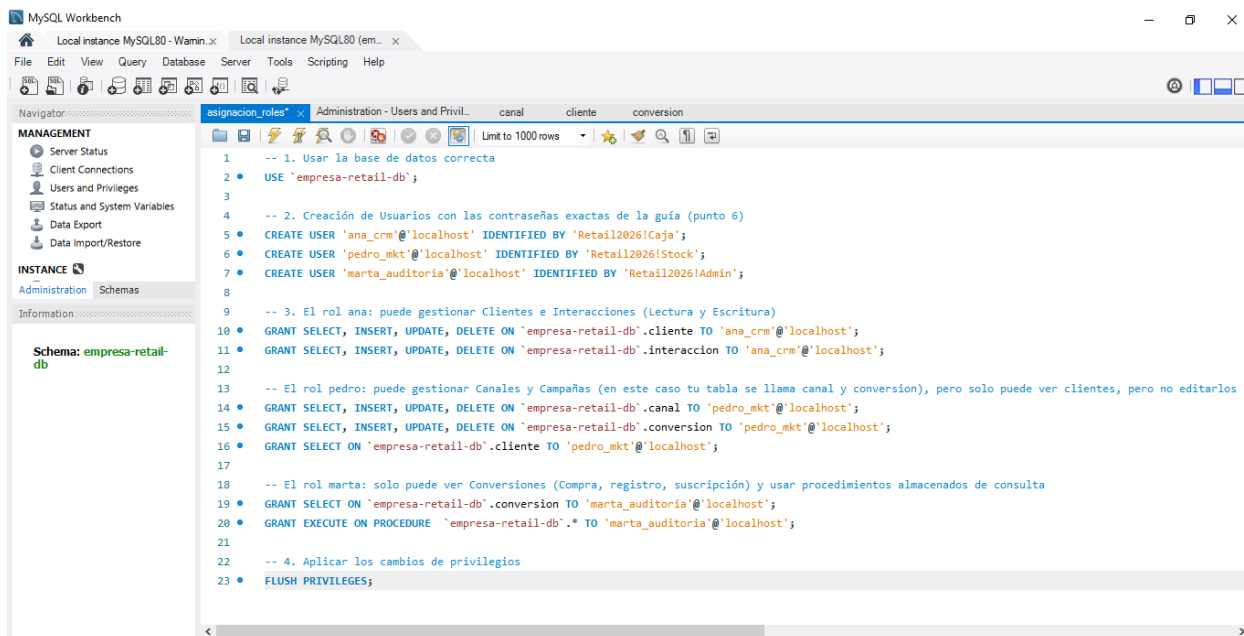
Nota. Captura de pantalla de elaboración propia de la rama develop en GitHub (interfaz en español). Se observan los archivos .gitignore, README.md y asignacion_roles.sql, el aviso de que la rama está dos confirmaciones por delante y una por detrás de la rama principal, y un total de tres confirmaciones. Fuente: GitHub.

4. Solución de la actividad en la rama develop

La solución se desarrolló y almacenó en la rama *develop*, como lo exige la actividad. Consistió en el script SQL `asignacion_roles.sql`, que se subió con el mensaje “Se sube el archivo de los roles” (Figura 3). El script se ejecutó en MySQL Workbench sobre la base de datos `empresa-retail-db` y se organiza en cuatro bloques: selección de la base de datos, creación de usuarios, asignación de permisos y aplicación de cambios. La Figura 4 presenta el script completo.

Figura 4

Script SQL de creación de usuarios y asignación de permisos



Nota. Captura de pantalla de elaboración propia del editor de consultas de MySQL Workbench con el script asignacion_rol.sql. Incluye USE, tres sentencias CREATE USER, las sentencias GRANT para cada usuario y FLUSH PRIVILEGES. Fuente: MySQL Workbench 8.0.

Creación de usuarios

Con la sentencia CREATE USER se crearon tres cuentas restringidas al host localhost, cada una con la contraseña indicada en la guía de la actividad (Oracle Corporation, s. f.-a). Las cuentas son ana_crm, pedro_mkt y marta_auditoria, que corresponden a las áreas de clientes, mercadeo y auditoría, respectivamente.

Asignación de permisos

Los permisos se definieron según la función de cada usuario mediante la sentencia GRANT (Oracle Corporation, s. f.-c). Ana gestiona la relación con los clientes, por lo que puede consultar, insertar, actualizar y eliminar registros en cliente e interaccion. Pedro administra las

acciones de mercadeo: tiene control sobre canal y conversion (tabla que en este esquema se asimiló a las campañas) y solo puede consultar cliente, sin editarla. Marta cumple un rol de auditoría: únicamente puede consultar conversion y ejecutar procedimientos almacenados de consulta. La Tabla 2 resume la asignación.

Tabla 2

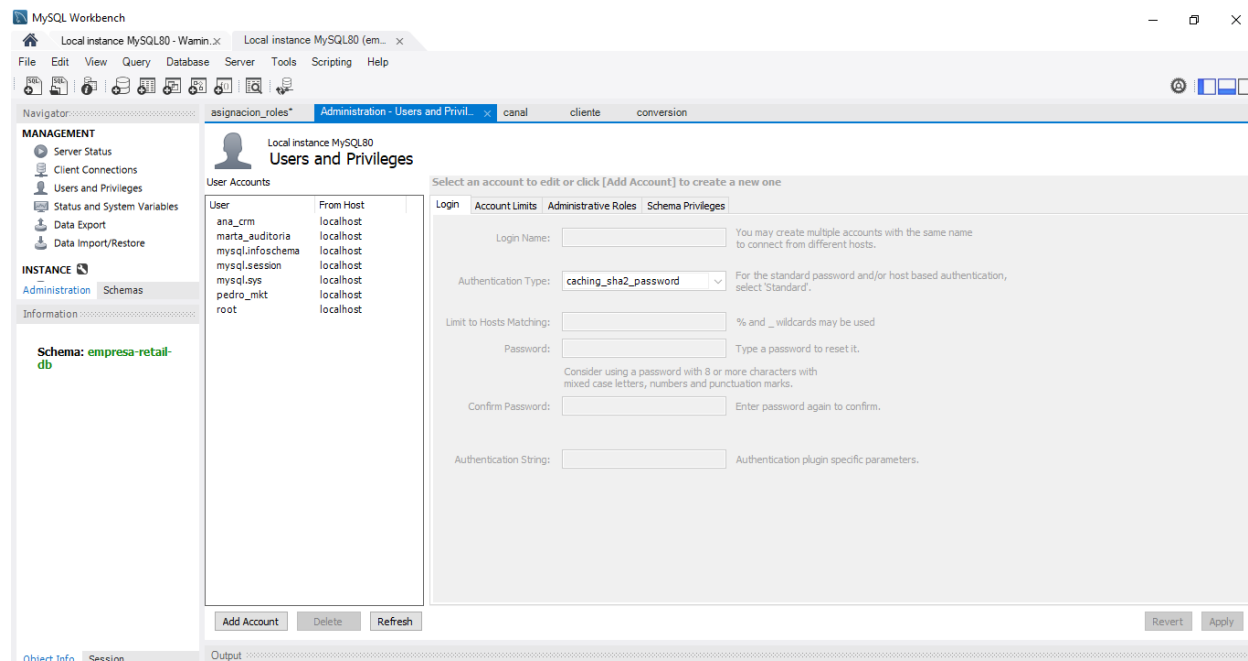
Resumen de usuarios, objetos y privilegios asignados

Usuario	Función	Objeto	Privilegios
ana_crm	Gestión de clientes e interacciones	cliente, interaccion	SELECT, INSERT, UPDATE, DELETE
pedro_mkt	Gestión de canales y conversiones de mercadeo	canal, conversion	SELECT, INSERT, UPDATE, DELETE
pedro_mkt	Consulta de clientes	cliente	SELECT
marta_auditoria	Auditoría de conversiones	conversion	SELECT
marta_auditoria	Consulta mediante procedimientos	Procedimientos de empresa-retail-db	EXECUTE

Nota. Elaboración propia con base en el script asignacion_rols.sql y en la guía de la actividad.

Verificación

Una vez ejecutado el script se verificó la creación de las cuentas en el módulo *Users and Privileges* de MySQL Workbench. Como se aprecia en la Figura 5, las cuentas ana_crm, marta_auditoria y pedro_mkt aparecen registradas con el host localhost junto con las cuentas del sistema, y utilizan el método de autenticación caching_sha2_password.

Figura 5*Verificación de los usuarios en MySQL Workbench*

Nota. Captura de pantalla de elaboración propia del módulo *Users and Privileges* de MySQL Workbench, con la lista de cuentas de usuario de la instancia local. Fuente: MySQL Workbench 8.0.

Análisis y recomendaciones de seguridad

La configuración cumple el objetivo de separar responsabilidades: ningún usuario recibió privilegios globales ni permisos de administración, y cada uno trabaja solo sobre los objetos de su área. Sin embargo, el análisis del script permite señalar los siguientes aspectos de mejora.

Permiso de ejecución sobre procedimientos. La sentencia `GRANT EXECUTE ON PROCEDURE` espera el nombre de un procedimiento concreto; si se escribe con comodín (*), MySQL interpreta el asterisco como el nombre de un procedimiento y devuelve el error 1305 (`PROCEDURE * does not exist`). Para conceder el permiso sobre todos los procedimientos del esquema se debe usar la forma a nivel de base de datos (Oracle Corporation, s. f.-c):

```
GRANT EXECUTE ON `empresa-retail-db`.* TO 'marta_auditoria'@'localhost';
```

Si se desea mayor control, es preferible otorgar EXECUTE procedimiento por procedimiento y solo sobre los de consulta, de modo que Marta no pueda ejecutar rutinas que modifiquen datos.

Restricción por tipo de conversión. Un GRANT SELECT sobre la tabla conversion permite ver todas sus filas, no solo las de compra, registro y suscripción. Si se requiere esa restricción, se puede crear una vista que filtre los tipos permitidos y otorgar el permiso únicamente sobre ella.

Sentencia FLUSH PRIVILEGES. En MySQL 8.0, las sentencias CREATE USER y GRANT actualizan los privilegios de inmediato; FLUSH PRIVILEGES solo es necesaria cuando las tablas de privilegios se modifican directamente (Oracle Corporation, s. f.-b). Su uso en el script es inofensivo, pero no indispensable.

Gestión de contraseñas. El script contiene contraseñas en texto claro y se publicó en un repositorio público. Esto es aceptable en un ejercicio académico con credenciales de ejemplo, pero en un entorno real las contraseñas nunca deben versionarse: deben generarse de forma segura, almacenarse fuera del repositorio y renovarse periódicamente.

Uso de roles. Como mejora, los permisos podrían agruparse en roles de MySQL (por ejemplo, un rol de gestión de clientes y otro de auditoría) y asignarse a los usuarios, lo que simplifica la administración cuando el número de cuentas crece (Oracle Corporation, s. f.-e).

Conclusión

La actividad permitió configurar la seguridad de la base de datos “empresa-retail” mediante la creación de tres usuarios con permisos diferenciados, aplicando el principio de mínimo privilegio: cada usuario accede solo a las tablas y operaciones que necesita en las áreas de clientes, mercadeo y auditoría. La verificación en MySQL Workbench confirmó la existencia

de las cuentas, y el análisis del script permitió identificar mejoras relacionadas con la sintaxis de los permisos de ejecución, el filtrado de datos mediante vistas y el manejo de contraseñas.

Asimismo, el uso de Git y GitHub aportó orden y trazabilidad al proceso. La creación del repositorio con las ramas *main* y *develop*, la documentación del proyecto en el README.md y la exclusión de archivos innecesarios con el .gitignore establecieron una base de trabajo profesional, y mantener la solución en la rama *develop* protege la versión estable hasta que los cambios sean revisados e integrados mediante una solicitud de extracción. Como trabajo futuro se recomienda utilizar roles de MySQL, evitar publicar contraseñas y revisar periódicamente los privilegios concedidos.

Referencias

- American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7.^a ed.). <https://doi.org/10.1037/0000165-000>
- Chacon, S. y Straub, B. (2014). *Pro Git* (2.^a ed.). Apress. <https://git-scm.com/book/es/v2>
- Git. (s. f.). *gitignore*. Git Documentation. <https://git-scm.com/docs/gitignore>
- GitHub. (s. f.-a). *About branches*. GitHub Docs. <https://docs.github.com/en/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/about-branches>
- GitHub. (s. f.-b). *About READMEs*. GitHub Docs. <https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes>
- GitHub. (s. f.-c). *Ignoring files*. GitHub Docs. <https://docs.github.com/en/get-started/git-basics/ignoring-files>
- GitHub. (s. f.-d). *Quickstart for repositories*. GitHub Docs. <https://docs.github.com/en/repositories/creating-and-managing-repositories/quickstart-for-repositories>
- Oracle Corporation. (s. f.-a). *CREATE USER statement*. En *MySQL 8.0 reference manual*. <https://dev.mysql.com/doc/refman/8.0/en/create-user.html>
- Oracle Corporation. (s. f.-b). *FLUSH statement*. En *MySQL 8.0 reference manual*. <https://dev.mysql.com/doc/refman/8.0/en/flush.html>
- Oracle Corporation. (s. f.-c). *GRANT statement*. En *MySQL 8.0 reference manual*. <https://dev.mysql.com/doc/refman/8.0/en/grant.html>

Oracle Corporation. (s. f.-d). *Privileges provided by MySQL*. En *MySQL 8.0 reference manual*.

<https://dev.mysql.com/doc/refman/8.0/en/privileges-provided.html>

Oracle Corporation. (s. f.-e). *Using roles*. En *MySQL 8.0 reference manual*.

<https://dev.mysql.com/doc/refman/8.0/en/roles.html>

Saltzer, J. H. y Schroeder, M. D. (1975). The protection of information in computer systems.

Proceedings of the IEEE, 63(9), 1278-1308. <https://doi.org/10.1109/PROC.1975.9939>